

# Terraform & OpenTofu IaC

Category: **Infrastructure as Code & Cloud Automation** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

---

## 1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Terraform & OpenTofu IaC**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

## 2. Core Concepts & Production Principles

- **Declarative Graph Resolution & Dependency Engine:** Terraform constructs a Directed Acyclic Graph (DAG) of all resources, determines implicit and explicit dependencies (`depends_on``), and provisions non-dependent resources in parallel.

*Production Application:* Enables rapid provisioning of complex environments while guaranteeing proper sequencing (e.g. creating VPC before subnets, subnets before instances).

- **State Management & Distributed Locking:** Terraform maps declarative code to real-world cloud resources via ``terraform.tfstate``. Remote backends (S3 + DynamoDB) prevent concurrent writes via distributed locks.

*Production Application:* Prevents catastrophic state corruption when multiple engineers or automated CI runners execute plans simultaneously.

- **Infrastructure Drift & Refresh Mechanics:** Discrepancy between the recorded state in ``terraform.tfstate`` and actual cloud infrastructure modified out-of-band via cloud consoles or CLI tools.

*Production Application:* Running automated daily ``terraform plan -detailed-exitcode`` in CI to detect and alert on unauthorized manual modifications.

- **Immutable Infrastructure & Replacement Lifecycles:** Replacing cloud instances or resources entirely rather than modifying them in-place, governed by lifecycle blocks (``create_before_destroy``, ``prevent_destroy``).

*Production Application:* Enforcing ``lifecycle { prevent_destroy = true }`` on production databases and storage buckets to prevent accidental deletion during refactoring.

## 3. Exhaustive Command & Syntax Reference

| Command & Syntax                                                                                                           | Purpose & Options                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Production Usage & Common Pitfall                                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>terraform plan</b><br><b>-out=tfplan.binary</b><br><b>-detailed-exitcode</b><br>terraform plan [options]                | <p>Generate an execution plan showing proposed infrastructure changes, saving it to a binary file and returning exit code 2 if changes exist.</p> <p><b>Options:</b> [{flag: '-out=', 'description': 'Save the generated execution plan to an immutable binary file.'}, {flag: '-detailed-exitcode', 'description': 'Returns code 0 on no changes, 1 on error, 2 if changes are present.'}, {flag: '-var-file=', 'description': 'Load variable definitions from external tfvars file.'}]</p> | <p>Run on every pull request to review proposed additions, modifications, and destructions before peer approval.</p> <p><b>Pitfall:</b> Applying plans without saving to an <code>-out`</code> file, leading to race conditions where the state changes between plan and apply.</p>                                       |
| <b>terraform apply</b><br><b>tfplan.binary</b><br>terraform apply [options]<br>[plan_file]                                 | <p>Apply the exact changes recorded in a pre-generated binary plan file without re-evaluating the graph or prompting for confirmation.</p> <p><b>Options:</b> [{flag: 'tfplan.binary', 'description': 'Execute the precise plan previously generated and validated during code review.'}, {flag: '-auto-approve', 'description': 'Skip interactive approval (only valid when applying without a plan file).'}]</p>                                                                           | <p>Execute strictly with pre-computed plan files in automated production deployment pipelines.</p> <p><b>Pitfall:</b> Running <code>`terraform apply -auto-approve`</code> directly against production code without reviewing a plan first.</p>                                                                           |
| <b>terraform state mv module.old_vpc.aws_subnet.public[0] module.networking.aws_subnet.public[0]</b><br>terraform state mv | <p>Move a resource within the Terraform state file to refactor module hierarchies without destroying and recreating cloud resources.</p> <p><b>Options:</b> [{flag: '', 'description': 'Current address of the resource in state.'}, {flag: '', 'description': 'New target address of the resource in state.'}]</p>                                                                                                                                                                          | <p>Execute when reorganizing code structure or renaming resource identifiers to prevent Terraform from destroying resources.</p> <p><b>Pitfall:</b> Renaming resource blocks in <code>.tf`</code> files without updating the state using <code>`state mv`</code>, causing Terraform to schedule resource destruction.</p> |
| <b>terraform import</b><br><b>aws_s3_bucket.data data-production-bucket-2026</b><br>terraform import [options]             | <p>Bring an existing cloud resource created manually or by legacy systems under Terraform state management.</p> <p><b>Options:</b> [{flag: '', 'description': 'Target HCL resource identifier declared in code.'}, {flag: '', 'description': 'Real-world cloud resource identifier (e.g. bucket name, VPC ID, ARN).'}]</p>                                                                                                                                                                   | <p>Declare the resource block in HCL, run import, then run <code>`terraform plan`</code> to align code parameters with live configuration.</p> <p><b>Pitfall:</b> Importing without matching HCL attributes, causing the subsequent plan to propose destructive updates.</p>                                              |

| Command & Syntax                                                     | Purpose & Options                                                                                                                                                                                                                                                                                                                          | Production Usage & Common Pitfall                                                                                                                                                                                                            |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>terraform force-unlock</b><br>terraform force-unlock<br>[options] | Manually release a stuck distributed state lock in DynamoDB or Terraform Cloud when a pipeline abruptly crashed.<br><b>Options:</b> [{flag: ", 'description': 'Unique lock identifier returned in the error message when state is locked.'}, {flag: '-force', 'description': 'Do not prompt for confirmation before releasing the lock.'}] | Execute ONLY after confirming that the previous pipeline job or developer has completely terminated.<br><b>Pitfall:</b> Unlocking state while another team member or pipeline job is actively applying changes, leading to state corruption. |

## 4. Security Hardening & Anti-Patterns

**Security Mandates:** Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.

**Critical Anti-Pattern:** Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.